

Thought for 10s >

Yes. This is now bigger than a Codex plugin.

The correct target is:

## BEAST × Obsidian 369 Phi IDE

### A full agent-native IDE, not a plugin pack

The strongest combined architecture is:

| Layer                | From BEAST / Byron                                               | From Obsidian 369 / Grant                                   | Combined result                                            |
|----------------------|------------------------------------------------------------------|-------------------------------------------------------------|------------------------------------------------------------|
| Execution governance | Action IR, provider routing, patch compiler, verifier, rollback  | Chainflow, TwinSwarm, mission lock                          | Governed autonomous coding inside an IDE                   |
| Agent orchestration  | Provider Economist, Tool Laziness, Session Handshake, MCP Broker | Tweedle-dee/Tweedle-dum, Obsidian Mirror, PulseOps          | Agents that plan, build, challenge, verify, log, and learn |
| IDE vision           | BEAST wanted to become a Visual Studio / Antigravity replacement | Obsidian wants default automatic Agent Builder cockpit      | Full AI-native IDE shell                                   |
| Memory               | Chronicle, PREC lifecycle, Memory Hull, local semantic cache     | Two-Brain Memory, Vault Cache, Echo Loop                    | Local memory system with active and reflective learning    |
| Safety/governance    | Bounded actions, schema validation, policy YAML, hash checks     | Tweety Canaries, Black Seal, Sphinx Gate, Light-Treat Light | Defensive local execution with audit and anomaly detection |
| Model routing        | OpenAI/Anthropic/HF/LiteLLM gateway, Provider Economist          | FCC, local LLM benchmarking, Codex integration              | Multi-provider + local LLM routing                         |
| UI/cockpit           | BEAST UI/TUI endpoints                                           | Obsidian visual dashboard and 369 doctrine                  | Full IDE workspace with live agent state                   |

BEAST already describes itself as sitting between coding agents, local tools, and model providers, giving models a small repository view, requiring bounded actions, compiling actions locally, verifying results, and recording what worked. That is exactly the execution core a full IDE needs.

BEAST also exposes OpenAI-compatible and Anthropic-compatible gateways plus BEAST-native governance endpoints, which means it can sit as the runtime engine behind external agents or a native IDE shell. Its coding flow endpoints include handoff preparation, governed source planning, execution, schema validation, maintenance checks, Chronicle, session handshake, and memory-security reporting.

---

# The merged product vision

## Name candidates

| Name                  | Best use                          |
|-----------------------|-----------------------------------|
| BEAST IDE             | Cleanest developer-facing product |
| Obsidian BEAST IDE    | Best combined brand               |
| BEAST 369 Workbench   | Best for Grant’s doctrine         |
| EdgeK Obsidian IDE    | Best if keeping EdgeK lineage     |
| BEAST Studio          | Best Visual Studio-style name     |
| Obsidian Agent Studio | Best for non-technical users      |

My recommendation:

## BEAST Studio

### Powered by Obsidian 369 Phi

Developer subtitle:

**A local-first agent-native IDE with governed execution, memory, audit, routing, verification, and autonomous workflow orchestration.**

---

## What this becomes

Not:

## Not this

---

A VS Code plugin only

---

A Codex wrapper only

---

A bunch of PowerShell scripts

---

A chatbot inside an editor

---

A loose swarm

---

A dashboard bolted onto a repo

---

It becomes:

## It becomes

---

A standalone IDE shell

---

A governed execution environment

---

A local AI workbench

---

A model/router/control plane

---

An agent builder

---

A memory/audit system

---

A code editor + terminal + agent cockpit

---

A VS Code / Antigravity competitor, over time

---

# Final architecture

## 1. IDE shell layer

| Component     | Function                                               |
|---------------|--------------------------------------------------------|
| Monaco editor | Code editor core, same editor engine family as VS Code |
| File explorer | Repo navigation                                        |

| Component         | Function                                                                                |
|-------------------|-----------------------------------------------------------------------------------------|
| Terminal panel    | PowerShell, Bash, Python, Node, Git                                                     |
| Agent panel       | Tweedle-dee, Tweedle-dum, BEAST agents, provider state                                  |
| Chainflow panel   | Observe → Map → Reverse Engineer → Build → Challenge<br>→ Verify → Log → Learn → Update |
| Diff/patch viewer | Shows proposed hunks before apply                                                       |
| Approval queue    | Send, merge, apply, run, update decisions                                               |
| Memory panel      | Active memory, reflective memory, Chronicle                                             |
| Route panel       | Provider Economist, local LLM routes, cost, latency, fitness                            |
| Audit panel       | Obsidian Mirror, Black Seal, Stone Ledger                                               |
| Canary panel      | Tweety alerts, anomaly triggers, Sphinx gates                                           |
| PulseOps panel    | Dawn Brief, Dusk Report, Night Watch                                                    |

## 2. BEAST execution spine

| BEAST capability     | IDE role                                 |
|----------------------|------------------------------------------|
| Action IR            | Agent output contract                    |
| Local patch compiler | Converts intent into deterministic edits |
| Schema validation    | Blocks malformed agent output            |
| Stale hash rejection | Blocks outdated edits                    |
| Provider Economist   | Chooses best model/provider per task     |
| Tool Laziness        | Prevents useless tool calls              |
| MCP Broker           | Governs local tool access                |
| Workspace graph      | Repo understanding                       |
| Chronicle            | Execution memory                         |

| BEAST capability   | IDE role                                   |
|--------------------|--------------------------------------------|
| Runtime governor   | Budgets, timeouts, circuit breakers        |
| Maintenance checks | Build, lint, syntax, tests                 |
| Session Handshake  | Tells every agent what BEAST already knows |

BEAST’s policy file already contains many of the system controls needed for a full IDE: protocol unification, context economy, MCP tool governance, runtime control, forensic observability, token caps, daily request/cost budgets, circuit breakers, workspace graph, MCP schema pinning, tool interception, Ollama scout, telemetry, file operation policies, and swarm roles.

### 3. Obsidian 369 layer

| Obsidian component   | IDE role                                   |
|----------------------|--------------------------------------------|
| 369 Mission Kernel   | Every task starts scoped                   |
| Tweedle TwinSwarm    | Builder/challenger loop                    |
| Obsidian Mirror      | Audit/reflection UI                        |
| Two-Brain Memory     | Active + reflective learning               |
| Black Seal           | Hash and integrity layer                   |
| Stone Ledger         | Backup and rollback                        |
| Sphinx Gate          | Intent challenge and risk classification   |
| Tweety Bird Canaries | Early-warning tripwires                    |
| Light-Treat Light    | Mismatch/anomaly detection                 |
| Kaleidoscope Mesh    | Visual symmetry/signature state            |
| PulseOps             | Scheduled work while away                  |
| Dawn/Dusk Reports    | IDE status intelligence                    |
| AI PA                | Drafting, email, research, admin workflows |

# How it works when opened

The user should not need to load plugins manually, run PS1 files manually, or remember setup steps.

## Startup sequence

| Step | What happens automatically                                            |
|------|-----------------------------------------------------------------------|
| 1    | User opens <b>BEAST Studio</b>                                        |
| 2    | IDE loads workspace/repo                                              |
| 3    | Local BEAST runtime starts                                            |
| 4    | LocalDB / Chronicle / Obsidian Mirror opens                           |
| 5    | Provider Economist checks available routes                            |
| 6    | Local LLM routes checked: Ollama/FCC/LM Studio/NIM as configured      |
| 7    | MCP Broker starts                                                     |
| 8    | Workspace graph indexes repo                                          |
| 9    | Black Seal hashes critical files                                      |
| 10   | Tweety Canaries arm                                                   |
| 11   | Chainflow panel becomes active                                        |
| 12   | Agents receive Session Handshake                                      |
| 13   | IDE shows operator dashboard                                          |
| 14   | User works normally, but the IDE is agent-aware from the first second |

## In VS Code / Codex / Antigravity compatibility mode

| External program | How it should work                               |
|------------------|--------------------------------------------------|
| VS Code          | Extension connects to local BEAST Studio runtime |
| Codex            | Uses local BEAST/Obsidian MCP + config           |

| External program | How it should work                                                             |
|------------------|--------------------------------------------------------------------------------|
| Antigravity      | Connects via local OpenAI-compatible/Anthropic-compatible gateway if supported |
| Claude Code      | Points to BEAST-compatible provider endpoint                                   |
| Cursor           | Points to BEAST gateway / MCP                                                  |
| Terminal         | CLI command starts or attaches to runtime                                      |
| Browser UI       | Local cockpit at localhost                                                     |

This gives two routes:

| Route       | Purpose                                                                        |
|-------------|--------------------------------------------------------------------------------|
| Native IDE  | Long-term full product                                                         |
| Attach Mode | Short-term compatibility with VS Code, Codex, Antigravity, Cursor, Claude Code |

## System phases to reach full IDE

### Phase 0: Joint architecture freeze

| Output                             | Purpose                   |
|------------------------------------|---------------------------|
| BEAST_OBSIDIAN_IDE_ARCHITECTURE.md | Single source of truth    |
| COMPONENT_MAP.md                   | BEAST ↔ Obsidian mapping  |
| ROADMAP.md                         | Phased implementation     |
| RISK_REGISTER.md                   | What not to run/build yet |
| MVP_SCOPE.md                       | Prevent overbuild         |

Time: **0.5–1 day**

Success rate: **95%**

### Phase 1: Runtime sidecar MVP

Goal: make BEAST run as a local sidecar that an editor can talk to.

| Build item             | Source           |
|------------------------|------------------|
| Health check           | BEAST            |
| Workspace index        | BEAST            |
| Context packet         | BEAST            |
| Policy loader          | BEAST            |
| MCP broker             | BEAST            |
| Audit log bridge       | BEAST + Obsidian |
| Mission packet wrapper | Obsidian         |
| Chainflow mapping      | Obsidian         |

Time: **2–4 days**

Success rate: **75–85%**

Output:

**Output**

Local runtime starts

Repo can be indexed

Policy loads

Basic MCP calls work

Mission packet can wrap a BEAST task

Logs write locally

**Phase 2: Editor integration shell**

Goal: use existing editor tech before building a custom IDE from scratch.



| Option                          | Rating | Notes                     |
|---------------------------------|--------|---------------------------|
| VS Code extension first         | 90%    | Fastest path              |
| Web IDE with Monaco             | 85%    | Best control              |
| Electron desktop shell          | 80%    | Best standalone app route |
| Full custom editor from scratch | 35%    | Too slow, not needed      |

Recommended route:

| Stage | Build                                               |
|-------|-----------------------------------------------------|
| 2A    | VS Code extension / Codex attach mode               |
| 2B    | Local web cockpit                                   |
| 2C    | Electron shell wrapping Monaco + terminal + cockpit |
| 2D    | Standalone BEAST Studio IDE                         |

Time: **1–3 weeks**

Success rate: **70–85%**

## Phase 3: Agent Builder inside IDE

Goal: make Agent Builder a native panel, not a prompt.

| Panel             | Function                   |
|-------------------|----------------------------|
| Agent Registry    | List agents, roles, routes |
| Mission Builder   | Create scoped missions     |
| Chainflow Runner  | Run 9-step workflow        |
| TwinSwarm Console | Builder/challenger output  |
| Provider Console  | Model route selection      |
| Tool Governance   | MCP evaluate/execute/audit |
| Skill Vault       | Create/install/rank skills |

| Panel          | Function                     |
|----------------|------------------------------|
| Approval Queue | Apply/reject/delay decisions |
| Memory Console | Active + reflective memory   |

Time: **2–4 weeks**

Success rate: **70–80%**

## Phase 4: Governed code execution

Goal: this is where it starts competing with real IDEs.

| Feature               | Role                              |
|-----------------------|-----------------------------------|
| Action IR viewer      | See exact model intent            |
| Source plan viewer    | See what BEAST thinks must change |
| Patch compiler        | Generate deterministic hunks      |
| Diff preview          | Select hunks                      |
| Hidden/visible verify | Run tests                         |
| Rollback              | Restore state                     |
| Chronicle             | Record result                     |
| Learning              | Store verified outcome            |

Time: **2–4 weeks**

Success rate: **65–80%**

## Phase 5: AI-native memory and local intelligence

| Feature                   | Source                      |
|---------------------------|-----------------------------|
| Workspace graph           | BEAST                       |
| Graphify-style visual map | Obsidian + Graphify concept |
| Active memory             | Obsidian Brain A            |

| Feature                | Source           |
|------------------------|------------------|
| Reflective memory      | Obsidian Brain B |
| Chronicle/PREC         | BEAST            |
| Local semantic cache   | BEAST            |
| Echo Loop              | Obsidian         |
| Skill improvement loop | Both             |

Time: **2–4 weeks**

Success rate: **65–78%**

## Phase 6: PulseOps and work-while-away

Goal: IDE does useful non-destructive work while the user is away.

| Job                | Output                     |
|--------------------|----------------------------|
| Night Watch        | Overnight repo/status scan |
| Dawn Brief         | Morning summary            |
| Dusk Report        | Evening summary            |
| Model benchmark    | Route fitness update       |
| Skill health check | Which skills worked/failed |
| Canary scan        | Suspicious events          |
| Repo risk scan     | Open failures/blockers     |
| Draft queue        | Draft-only outputs         |

Time: **1–3 weeks**

Success rate: **70–85%**

## Phase 7: Standalone BEAST Studio IDE

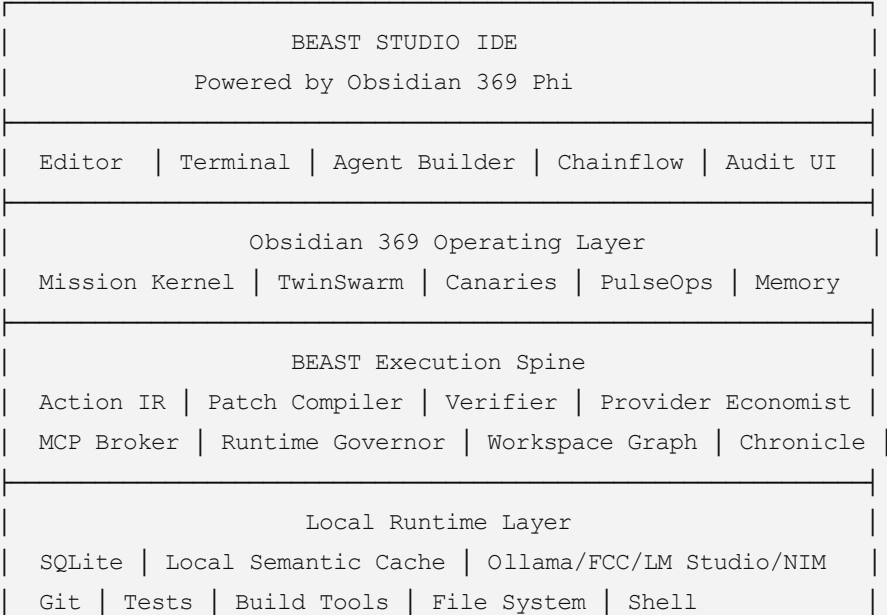
Goal: no more “load plugin, run PS1, open terminal”.

| Component        | Build choice               |
|------------------|----------------------------|
| Desktop shell    | Electron or Tauri          |
| Editor           | Monaco                     |
| Terminal         | xterm.js / native PTY      |
| Runtime          | BEAST local server         |
| Database         | SQLite/local stores        |
| Agent UI         | React                      |
| MCP              | Local bridge               |
| Provider gateway | BEAST                      |
| Installer        | Windows first              |
| Auto-start       | App-managed service        |
| Updates          | Signed local updater later |

Time: **2–4 months for v1**

Success rate: **60–75%**

## Full IDE architecture



|                                  |           |            |    |      |        |
|----------------------------------|-----------|------------|----|------|--------|
| Optional External Provider Layer |           |            |    |      |        |
| OpenAI                           | Anthropic | OpenRouter | HF | Groq | Gemini |

## What should be automatic on open

| Feature                      | Auto on open? | Notes                              |
|------------------------------|---------------|------------------------------------|
| Runtime health check         | Yes           | Non-destructive                    |
| Workspace graph state        | Yes           | Read-only                          |
| Provider route status        | Yes           | No provider call unless needed     |
| LocalDB open                 | Yes           | Local only                         |
| Chronicle open               | Yes           | Local only                         |
| Canaries arm                 | Yes           | Defensive                          |
| Mission restore              | Yes           | Load last active mission           |
| Terminal attach              | Yes           | Integrated                         |
| Codex/Agent Builder doctrine | Yes           | Loaded from project config         |
| PulseOps status              | Yes           | Show queued items                  |
| LLM benchmark                | No            | Manual or scheduled                |
| External API call            | No            | Approval or task-triggered         |
| Code patch                   | No            | Only after mission/action approval |
| Email send/post/merge/delete | No            | Never automatic by default         |

## What Byron should take from Grant's design

| Grant layer                 | Why it matters for BEAST IDE             |
|-----------------------------|------------------------------------------|
| Obsidian 369 brand/doctrine | Gives BEAST a memorable product identity |

| Grant layer               | Why it matters for BEAST IDE                         |
|---------------------------|------------------------------------------------------|
| TwinSwarm                 | Turns verification into visible adversarial workflow |
| Chainflow                 | Makes execution explainable to users                 |
| Two-Brain Memory          | Separates working context from long-term learning    |
| Tweety Canaries           | Adds memorable early-warning system                  |
| Light-Treat Light         | Gives anomaly detection a visual logic               |
| PulseOps                  | Adds work-while-away usefulness                      |
| AI PA / postgrad/research | Expands beyond coding into broader productivity      |
| Visual architecture       | Makes the system sellable and teachable              |

## What Grant should take from BEAST

| BEAST layer        | Why it matters for Obsidian 369                  |
|--------------------|--------------------------------------------------|
| Action IR          | Prevents uncontrolled agent edits                |
| Patch compiler     | Makes code changes deterministic                 |
| Provider Economist | Makes model routing evidence-based               |
| Tool Laziness      | Stops tool spam                                  |
| MCP Broker         | Gives real governance to local tools             |
| Workspace graph    | Gives repo understanding                         |
| Runtime Governor   | Gives budgets/circuit breakers                   |
| Chronicle/PREC     | Gives real memory/audit structure                |
| Policy YAML        | Gives operational control                        |
| Cockpit/TUI        | Gives UI direction                               |
| Compute Governor   | Saves tokens and reduces unnecessary model calls |

# Recommended MVP

Do not try to build the whole IDE first.

## MVP name

### BEAST Studio Attach Mode

Purpose: prove the runtime and workflow inside existing VS Code/Codex before building standalone.

| MVP feature             | Include                |
|-------------------------|------------------------|
| Local BEAST sidecar     | Yes                    |
| Obsidian mission packet | Yes                    |
| Chainflow panel or CLI  | Yes                    |
| MCP read-only tools     | Yes                    |
| Action IR viewer        | Yes                    |
| Diff preview            | Yes                    |
| Audit log               | Yes                    |
| Provider route state    | Yes                    |
| Canaries                | Basic                  |
| PulseOps                | Basic Dawn/Dusk report |
| Standalone IDE shell    | No                     |
| Full dashboard          | No                     |
| Marketplace             | No                     |
| External actions        | No                     |

Time: **2–4 weeks**

Success: **75–85%**

---

# Full product timeline

| Stage                      | Duration     | Maturity |
|----------------------------|--------------|----------|
| Architecture freeze        | 1 day        | 10%      |
| Runtime sidecar            | 1 week       | 25%      |
| VS Code/Codex attach mode  | 2–4 weeks    | 40%      |
| Agent Builder cockpit      | 1–2 months   | 55%      |
| Governed code execution UI | 2–3 months   | 70%      |
| Standalone desktop shell   | 3–5 months   | 80%      |
| Polished BEAST Studio v1   | 6–9 months   | 90%      |
| Enterprise-grade IDE       | 12–18 months | 95%      |

# Success probability

| Target                                       | Probability                 |
|----------------------------------------------|-----------------------------|
| Attach mode working in VS Code/Codex         | 80–88%                      |
| Usable local agent-builder cockpit           | 72–82%                      |
| Standalone IDE prototype                     | 65–75%                      |
| Full Visual Studio / Antigravity replacement | 45–60%                      |
| Commercial-grade IDE                         | 35–50% without team/funding |
| Commercial-grade IDE with 3–5 devs           | 65–80%                      |

# Final strategic decision

The best architecture is:

**Obsidian = operating doctrine and user experience**



**BEAST = governed execution engine**

**BEAST Studio = full IDE product**

Do not bury this inside plugins forever. Use plugins and attach mode as the bridge. The final form is a standalone IDE.

The build path should be:

1. **BEAST sidecar runtime**
2. **VS Code/Codex attach mode**
3. **Obsidian Agent Builder cockpit**
4. **Governed code execution panels**
5. **Standalone Electron/Tauri IDE**
6. **Commercial BEAST Studio**

Final verdict:

**Yes, full-blown IDE is the right direction.**

But it should not start as a Visual Studio replacement. It should start as an **agent-native execution cockpit attached to VS Code/Codex**, then graduate into **BEAST Studio**, a standalone IDE where the agent builder, terminal, editor, memory, audit, model routing, and governed execution are all native.